MODULE 3

From Trained Model to Local API

FastAPI & Docker Containerization

Beginner lab guide

Docker Desktop setup · saved model weights · preprocessing · FastAPI · Docker containers

Before class: allow 30-60 minutes to prepare and verify Docker.
In class: 90-minute guided API and container workshop.
Operating systems: Windows and macOS instructions included.

UNIVERSITY OF THE PACIFIC · MSBA 265

Educational demonstration only. Not a medical diagnostic tool.

How to use this guide

Work through these three parts in order. First, install Docker Desktop and verify that its engine can run a sample container. Next, inspect the provided model handoff and learn what is saved besides weights. Then complete the guided FastAPI lab and run the service in Docker.

Do not skip computer setup. Docker installation, updates, and image downloads can take longer than the class. If an administrator or device policy blocks installation, contact the instructor before class; do not bypass the policy.

Contents

1. Docker Desktop setup pre-lab
2. Model handoff: weights and preprocessing
3. 90-minute guided FastAPI and Docker lab

DOCKER DESKTOP SETUP PRE-LAB

Before the Lab: Install Docker Desktop

Audience: first-time Docker users on Windows or macOS. **Allow:** 30-60 minutes before class; updates, restarts, and campus support may take longer. Do not begin installation during the timed workshop.

What is Docker, and is it free?

Imagine handing someone a recipe, ingredients, and the right kitchen setup instead of only your finished meal. Docker packages an application's software environment so it can run consistently on another compatible computer. It does not magically fix code, replace a trained model, or make predictions more accurate.

- **Docker Desktop:** the application you install on your laptop. It provides tools and a local engine for running containers.
- **Image:** a packaged application template; not a photo.
- **Container:** a running instance of an image.
- **Dockerfile:** the recipe used to build the image.
- **Docker Hub:** an online image registry. It is different from GitHub, which hosts our source repository.

Yes: Docker Desktop is free for educational use, according to Docker's license guidance. Students do not need to buy a paid plan for this local class lab. This does not mean every workplace qualifies for free use; commercial/government rules differ. Cloud hosting and paid Docker features are not required. Read the terms presented by your installer. Do not enter payment information for this exercise. You do not need to publish an image to Docker Hub. Some networks or pull limits may require a free account; a school-managed installation may require its organization's sign-in.

1. Identify your computer

Windows

1. Open **Start -> Settings -> System -> About**. Record the Windows edition, version, system type, and installed memory.
2. Most Windows laptops are x64/AMD64, including Intel processors. ARM devices need an ARM-compatible Docker installer and may have different support.
3. Open **Task Manager -> Performance -> CPU** and look for **Virtualization: Enabled**.
4. Compare your computer with Docker's current Windows requirements. The WSL 2 x64 path lists WSL 2.1.5 or later, 8 GB RAM, supported 64-bit Windows, and hardware virtualization. OS support changes; use that official page rather than assuming every Windows release is supported.

If virtualization is disabled or installation is blocked on a managed device, ask campus IT or the instructor. Do not change firmware settings, disable security software, or bypass school policies on your own.

Mac

1. Open **Apple menu -> About This Mac**.
2. Record the macOS version and whether it says **Chip: Apple M...** (Apple silicon) or **Processor: Intel**.
3. Compare with Docker's current Mac requirements. Docker lists at least 4 GB RAM and supports the current and two previous major macOS releases. More available memory is useful for the lab.
4. Leave several GB of free disk space for Docker and downloaded images; the precise requirement depends on images and software updates.

2A. Install on Windows using WSL 2

WSL means **Windows Subsystem for Linux**. Docker uses it to run the Linux environment needed by our container; you do not need to replace Windows.

1. Open **PowerShell** from Start. Run:

```
wsl --version
wsl --status
```

2. If WSL is missing, follow Microsoft's WSL installation instructions. On your own authorized computer, open PowerShell **as Administrator** and run:

```
wsl --install
```

Restart when instructed. If a Linux distribution asks you to create a username/password, complete that setup; its password entry may show no characters as you type. Keep the credentials private.

3. If WSL needs updating, follow the same official guidance and run:

```
wsl --update
wsl --version
```

Ask IT if permissions or Windows features block this. Do not repeatedly reinstall WSL or remove distributions containing your files.

4. Download **Docker Desktop for Windows** from the official Docker Windows installation page. Choose the installer matching your system architecture.
5. Run the installer and use the **WSL 2** backend when offered. Accept the license only after reading it. Follow any restart prompt.
6. Open **Docker Desktop** from Start. Complete onboarding without purchasing a subscription. Let the engine finish starting.
7. In Docker Desktop settings, verify **Use the WSL 2 based engine** if that option is available. Our lab needs **Linux containers**, not Windows containers. Do not change to Windows containers.
8. Close and reopen an ordinary PowerShell window, then continue to Step 3. You do not need to run everyday lab commands as Administrator.

2B. Install on Mac

1. Open Docker's official Mac installation page and choose **Apple silicon** or **Intel** based on Step 1. Do not choose by guessing.
2. Open the downloaded `Docker.dmg` and drag Docker into **Applications**.
3. Open **Applications -> Docker**. Read/accept its license and follow the installer prompts. Ask IT about any administrator restrictions.
4. Complete onboarding without purchasing a subscription. Wait for the engine to be running. Leave Docker Desktop open in the background.
5. Open **Terminal** using Command+Space -> type Terminal -> Return.
6. Continue to Step 3. Rosetta is not a prerequisite for our native multi-architecture Python image; if another Docker tool requests it, follow Docker's official guidance rather than installing unrelated software.

3. Prove that installation works (both systems)

In PowerShell or Terminal, run one line at a time:

```
docker --version
```

```
docker version
docker run --rm hello-world
```

Expected results:

- `docker --version` prints the installed command-line tool's version.
- `docker version` prints **both Client and Server** information. Client information alone does not prove the engine is running.
- `docker run --rm hello-world` downloads a tiny demonstration image and prints **Hello from Docker!**. The demonstration container then exits and is removed. It does not start our model service.

The first pull requires internet access. Do not paste private account tokens or your full system information into public Git issues.

If a check fails

Problem	Next step
`docker` is not recognized / command not found	Finish installation, reopen the terminal, and check Docker's official PATH/setup instructions.
Cannot connect to daemon/engine; named pipe or socket error	Open Docker Desktop and wait until the engine is running. On Windows confirm WSL readiness and Linux-container mode.
WSL or virtualization error	Capture the error and ask IT/instructor; do not bypass device-management policy.
`hello-world` cannot download, or a pull limit appears	Check internet/proxy settings and the Docker message; sign in with a free account if required. Do not buy a plan for the lab.
Unsupported OS or insufficient hardware	Tell the instructor before class. Arrange an approved supported lab computer; do not claim a completed Docker run.

4. Bring your readiness evidence

Record your OS/chip, Docker client/server versions, and whether `Hello from Docker!` appeared. An optional cropped screenshot may show only the success message, not personal information.

You are ready when Docker Desktop runs and all three checks succeed. Then return to the guided workshop. Python/Git setup is also required; Docker installation alone does not install the host Python tools.

The class goal includes building/running the model container, not just installing the Docker command. If your setup is blocked, notify the instructor and complete the native API activities while an approved solution is arranged.

MODEL HANDOFF: WEIGHTS AND PREPROCESSING

Model Handoff: What Would You Save After Training?

A folder, not just a mysterious "weights file"

After training, you should be able to hand another person enough information to reproduce **inference** without rerunning training. This workshop gives you that handoff in artifacts/.

Cloning or downloading the course repository downloads the folder. Alternatively, download model_handoff.zip from the course repository or the copy supplied by your instructor. Extract it into this workshop folder: the resulting `artifacts` folder must sit beside app.py. Do not end up with `artifacts/artifacts`. If your browser saves the ZIP in Downloads, move it to the workshop folder before extracting it.

On Windows, right-click the ZIP -> **Extract All** and select the workshop folder. On macOS, double-click the ZIP and move its `artifacts` folder beside the API script if Finder creates an additional enclosing folder.

The ZIP is a model pack, not the complete application. You still need the workshop source code and dependencies. Keep the supplied files together. Open these JSON files in a text editor; do not edit their numbers for the lab.

Supplied item	What it means	What you would save from your own trained model
`logistic_weights.json`	Two learned coefficients, an intercept, class ordering, and threshold; coefficients apply to standardized inputs	The model's `coef_`, `intercept_`, `classes_`, and documented decision rule
`preprocessing.json`	Training means/scales and required feature order	The fitted scaler's `mean_` and `scale_`; also fitted encoders/imputers if your model uses them
`teaching_pipeline.joblib`	Reloadable fitted scaler and logistic estimator together	A serialized fitted preprocessing/model pipeline
`model_metadata.json`	Model ID, source/context, feature contract, package versions, and reference output	A record of how to load/use the model and which training run it belongs to
`example_request.json`	A valid sample input	A documented input/output example for checking a handoff

The plain JSON files make the learned values visible. The API loads the saved **pipeline** to preserve scikit-learn's preprocessing and prediction behavior. It does not load JSON weights into a second independently fitted model. Changing a JSON number alone will not change that serialized pipeline. Rebuild and verify the whole pack if intentionally changing the model.

Security: load joblib/pickle files only from a trusted source. A serialized Python object can execute code during loading. A file extension or classmate's claim is not a security guarantee. The API has no file-upload endpoint.

Connect weights to the equation you know

The raw inputs are `mean_radius` and `mean_texture`. Before applying the logistic coefficients, the saved scaler computes:

```
x1_scaled = (mean_radius  - training_mean[0]) / training_scale[0]
x2_scaled = (mean_texture - training_mean[1]) / training_scale[1]
z = intercept + coefficient[0]*x1_scaled + coefficient[1]*x2_scaled
```

```
p = 1 / (1 + exp(-z))
```

The intercept is the constant term, sometimes called the bias. The two coefficients are the feature weights. They are learned during training; the API merely reuses them. A threshold is a **decision policy**, not an additional learned feature coefficient. Our 0.50 threshold is fixed for this demonstration, not presented as a clinically validated operating point.

Inspect-and-explain activity

Before starting the API:

1. Open the weights JSON and identify the intercept and each feature's weight.
2. Open the preprocessing JSON and find the two training means and scales.
3. Explain why a raw radius of 14 must not be multiplied directly by a standardized-feature coefficient without first applying the saved scaler.
4. Open the metadata and find the class-1 meaning, threshold, build versions, and reference probability.
5. Identify which file(s) let a teammate run inference, and why weights alone would not communicate feature order, scaling, and class meaning reliably.

The unit test `test_readable_weights_and_scaler_reproduce_pipeline` independently calculates probabilities from these readable values and checks them against the saved pipeline. It is a consistency check, not a model-accuracy evaluation.

If you trained your own model later

The essential pattern for a scikit-learn model is:

```
pipeline.fit(X_train, y_train)
joblib.dump(pipeline, "teaching_pipeline.joblib")
```

Fit using training data only, finish your evaluation decisions, and save the fitted pipeline without refitting it on API requests. Also save a feature schema, class meanings, preprocessing details, library versions, chosen threshold, source/training-run information, and a reference request/response. See prepare_model.py for this pack's complete export process; running it is **not required** for students receiving the supplied artifact.

A neural network's weights file additionally needs its architecture and matching preprocessing/code. A tree model saves split structures rather than logistic coefficients. "Save the weights" is not a universal complete deployment format.

Do not save private training records, passwords, API keys, or peer signatures inside a model pack. This teaching artifact is deliberately small and has no claim of clinical validity. Do not substitute it for Project 2's required fit.

90-MINUTE GUIDED FASTAPI AND DOCKER LAB

Guided Workshop: From a Trained Logistic Model to a Local API

Duration: 90 minutes after software setup. **Goal:** use a supplied trained model, not train a new one. **Safety:** classroom example only; not a medical diagnostic application.

Before class, complete the Docker Desktop setup guide, including the client/server and hello-world checks. Docker Desktop is free for educational use under its current terms; no paid cloud hosting is used here. Review the model handoff guide to learn what the supplied weights, preprocessing values, and fitted pipeline represent.

1. Before class: get the software and project

You need internet access for installation, approximately 1 GB of free space for Python/packages, and more space if using Docker. This workshop uses **standard Python 3.14**, matching the supplied artifact's environment; do not choose the experimental free-threaded build. An editor is optional.

- Install Python from python.org. On Windows, enable the Python command/PATH option if the installer offers it. On macOS, complete the installer, including its certificate setup if prompted.
- Install Git from git-scm.com.
- Close and reopen your terminal after installing software.
- Docker setup is a required pre-lab. Follow the separate step-by-step guide for your OS and processor; start Docker Desktop and verify that its engine can run a container before class.

Open a terminal: on Windows, search Start for **PowerShell**. On a Mac, press Command+Space, type **Terminal**, and press Return. A terminal accepts typed commands; press Enter/Return after each line. Do not type the Markdown backticks surrounding examples.

If you already have this course repository, open its root folder in your terminal and skip cloning. Otherwise, the following commands create a folder named `MSBA265` in your terminal's current directory. If the instructor provides a workshop branch, switch to that branch before continuing.

Windows PowerShell

```
git --version
py -3.14 --version
git clone https://github.com/shylasolis/MSBA265.git
cd .\MSBA265\Module_03_FastAPI_Docker\workshop
py -3.14 -m venv .venv
.\.venv\Scripts\python.exe -m pip install -r requirements-dev.txt
```

These Windows commands call the environment's Python directly, so activation and PowerShell execution-policy changes are unnecessary.

macOS Terminal

```
git --version
python3.14 --version
git clone https://github.com/shylasolis/MSBA265.git
cd MSBA265/Module_03_FastAPI_Docker/workshop
python3.14 -m venv .venv
./.venv/bin/python -m pip install -r requirements-dev.txt
```

If starting from the existing course repository root, navigate only into `Module_03_FastAPI_Docker` and then `workshop` using your OS's path convention. All remaining commands assume that **workshop is your current folder**. Installation may take several minutes. Continue only if it finishes without errors. Pinned primary package versions are in requirements.txt; the development file adds the API test client.

2. Understand the supplied model (minutes 0-25)

Training adjusts model coefficients using labeled examples. **Inference** uses the already learned coefficients on new measurements. An **artifact** is a saved training result that another process can load.

Open model_metadata.json in a text editor. It documents the source, feature order, threshold, package versions, and a reference request. The paired teaching_pipeline.joblib contains:

```
raw measurements -> fitted StandardScaler -> fitted LogisticRegression
```

Do not open the binary joblib file as text. Do not replace it with an artifact from an unfamiliar source: joblib/pickle can execute code during loading. The service does not accept uploaded model files.

The two features, in order, are `mean radius` and `mean texture` from scikit-learn's bundled Wisconsin Diagnostic Breast Cancer dataset. Radius is a cell-nucleus size measurement and texture describes grayscale variation; these are not a person's age or body temperature. The API field names use underscores: `mean_radius`, then `mean_texture`.

The provided model defines **1 = malignant** and **0 = benign**. This differs from the bundled dataset's original coding. The probability is for class 1. The decision rule is `p >= 0.50` -> class 1; otherwise class 0.

The training-only scaler and coefficients are already fitted. **Never fit a scaler on a single API request.** The saved pipeline transforms new raw features using the training means and scales. Sending already standardized inputs would scale them twice and produce incorrect predictions.

prepare_model.py documents how the instructor built the artifact. You do **not** need to run it, finish Project 2, or download any data. This demo uses a regularized model and fixed threshold; it is not Project 2's training or evaluation experiment.

Discuss: which files contain learned values, which contain instructions, and why must the feature order remain consistent?

3. Start the service (minutes 25-40)

An **API** is an interface that lets programs exchange requests and responses. FastAPI defines the interface; Uvicorn is the server process that listens for requests.

Think of the browser as a customer, a JSON request as an order, the API as the agreed ordering format, and the saved model as the calculation behind the counter. The server waits for orders; it does not retrain the model for each one.

First-time API vocabulary	Meaning in this lab
Client	Your browser, or the Python checker sending a request
Server	The running Uvicorn process hosting the FastAPI application
Endpoint	A named API route such as `/health` or `/predict`
Request body	The JSON input measurements sent with POST
Response	JSON returned by the service
Status code	HTTP result indicator: 200 for success, 422 for invalid input

First-time API vocabulary	Meaning in this lab
Input schema	Required names/types/limits described by `PredictionRequest`
`/docs`	FastAPI's generated interactive interface for trying requests

Unlike the Matplotlib GUI, this API has no sliders or prediction chart. `/docs` supplies a convenient test screen; another program can use the same endpoint without that screen. Opening a Python file is not the same as starting its server.

In your workshop terminal, run the command for your OS:

Windows:

```
.\.venv\Scripts\python.exe -m uvicorn app:app --host 127.0.0.1 --port 8000
```

macOS:

```
./.venv/bin/python -m uvicorn app:app --host 127.0.0.1 --port 8000
```

`app:app` means the `app` object inside app.py. `127.0.0.1` is your own computer; `8000` is the port, a numbered connection point. This local lesson does not expose the service to the public internet.

Wait for **Application startup complete**. The terminal remains busy because the server is running; this is expected. Leave it open.

In a web browser, open:

- `http://127.0.0.1:8000/health` to see a ready status and model ID.
- `http://127.0.0.1:8000/docs` to use the interactive API documentation.

The documentation screen may need internet access to load its interface assets; the prediction endpoint itself does not use an external prediction service.

4. Send a prediction (minutes 40-55)

In `/docs`, expand **POST /predict**, click **Try it out**, replace the example request with the following JSON, then click **Execute**:

```
{
  "mean_radius": 14.0,
  "mean_texture": 20.0
}
```

JSON uses double-quoted field names, colons between keys and values, and commas between fields. Numbers are not quoted. **POST** sends a body to the endpoint; **GET** retrieves a resource, such as health information.

The response should have HTTP status **200** and approximately:

```
{
  "model_id": "logistic-teaching-v1",
  "probability_class_1": 0.3265635181,
  "predicted_class": 0,
  "label": "benign",
  "threshold": 0.5,
  "positive_class": "malignant",
  "educational_only": true
}
```

Tiny floating-point differences are acceptable. This is a model output for a classroom example, not a diagnosis. A successful response does not establish accuracy or clinical validity.

Trace the request in app.py: schema validation -> raw feature array in fixed order -> saved pipeline -> class-1 probability -> threshold -> response. The model loads once at startup, not once per request, and there is no training endpoint.

5. Reject invalid inputs and verify results (minutes 55-65)

Try each of these requests separately in `/docs`:

1. Remove `mean_texture`.
2. Set `mean_radius` to `-1`.
3. Send `"14"` as text instead of `14` as a number.
4. Add an extra field such as `"age": 50`.

Each should return **422**, with details about which input failed validation. Restore the valid request and confirm it returns 200 again. These errors are part of the interface contract, not evidence that the model retrained. The demonstration accepts finite numeric values greater than 0 and no more than 100 for either feature. These are simple classroom guardrails, not validated biological limits or a check for realistic/in-distribution data.

Open a **second terminal**, navigate to the same workshop folder, and keep the server running in the first terminal. Execute:

Windows:

```
.\.venv\Scripts\python.exe check_api.py
.\.venv\Scripts\python.exe -m unittest discover -s . -p "test_*.py" -v
```

macOS:

```
./.venv/bin/python check_api.py
./.venv/bin/python -m unittest discover -s . -p "test_*.py" -v
```

`check_api.py` sends real HTTP requests to the server and compares the prediction to the supplied metadata. Expect `PASS: health and prediction match the supplied reference.` The unit tests use an in-process client and do not need the server; they check the saved pipeline, invalid requests, threshold equality, and that inference does not refit or change the scaler.

The server normally starts in seconds once dependencies are installed. Training is not part of request latency. This workshop prints responses; it does not create Project 2's evaluation CSVs or overwrite its results.

6. Package and run the service in Docker (minutes 65-85)

An **image** packages the application and its dependencies. A **container** is a running instance of that image. Dockerfile describes how to build the image; .dockerignore prevents local environments and caches from entering the build context. The trained artifact is copied, not retrained.

Complete this activity after the Docker readiness pre-lab. These commands are the same on both operating systems. If setup remains blocked, practice the native API and arrange an approved supported computer with the instructor; the module's Docker checkpoint remains incomplete until you run it. First stop the native server with **Ctrl+C** in its terminal. From the workshop folder:

```
docker version
docker build -t msba265-logistic-api .
docker run --rm --name msba265-logistic-workshop -p 127.0.0.1:8000:8000 msba265-logistic-api
```

The first build downloads a Python base image and packages and can take several minutes. Inside the container, Uvicorn binds to `0.0.0.0` so the port can be reached through Docker. The host mapping binds only to `127.0.0.1`, keeping the lesson local. `EXPOSE 8000` documents a port; it does not publish it by itself. The process runs as an unprivileged user.

Repeat `/health`, the `/docs` prediction, and `check_api.py` from the host's second terminal. The reference prediction should not change. Stop the container with Ctrl+C, or in another terminal use:

```
docker stop msba265-logistic-workshop
```

`--rm` removes this stopped container, not your source files or the image. Do not run broad container/image cleanup commands on a shared machine.

7. Exit checkpoint (minutes 85-90)

Keep brief notes for class discussion; this is not a new graded project.

1. Explain training versus inference and identify what is saved.
2. Explain why the scaler must travel with the model and why feature order matters.
3. Show a valid response and a 422 response; explain the difference.
4. Describe the full request path without saying that the API "trains a prediction."
5. Explain image versus container, and what `127.0.0.1:8000:8000` maps.
6. Name two reasons this local service is not production-ready.

Also retain the readiness check and evidence that the **container**, not just the native Python process, returned the reference prediction. If the Docker build is slow, finish it in follow-up practice rather than claiming it ran.

For portfolio integration later, retain professional Git organization, beginner Windows/macOS instructions, and independent peer validation. Any completed signed course validation form belongs on Canvas, not in Git.

Troubleshooting

Symptom	What to check
`git`, `py`, or `python3.14` not found	Install the named software, reopen the terminal, and verify its version. On Windows, if the launcher is unavailable but `python --version` shows 3.14, use `python -m venv .venv` for creation.
No `requirements-dev.txt` or cannot import `app`	You must be inside `Module_03_FastAPI_Docker/workshop`, not the repository root. Check the folder name in your terminal prompt.
`ModuleNotFoundError`	Install dependencies with the same `.venv` Python used in the run command. Do not use a different system Python to launch Uvicorn.
Missing artifact or version mismatch	Restore the supplied artifact/metadata pair and install the pinned requirements. Do not bypass the version check or substitute an unknown joblib file.
Address already in use	Stop your previous workshop server/container. Do not terminate unrelated processes. If another user owns port 8000, ask the instructor for a different port and update the URLs, checker, and Docker host mapping together.
Browser connection refused or checker fails to connect	Keep the server terminal open; read its startup error and confirm the host/port. If startup failed, correct the error before proceeding.
`/` gives 404 or browser `/predict` gives 405	There is no home page; use `/docs`. Typing `/predict` in the address bar sends GET, but prediction requires POST.
Docker cannot connect to its engine	Start Docker Desktop, wait for readiness, and check Linux-container mode. Continue the native workshop while installation is resolved.
Docker build fails on a package or base-image download	Check internet access and Docker's connection. Capture the error; do not claim a successful container run.

Verification

The supplied model was built with Python 3.14.3 and the primary versions pinned in requirements.txt. The saved model records its build versions in the metadata. Pinning these libraries improves reproducibility; it is not a complete cross-platform lockfile or a guarantee of clinical validity.

Executed on the local Windows Python 3.14.3 environment:

- All six unittest cases passed, including readable-weights/scaler equivalence, invalid input rejection, threshold equality, and no refitting during requests.
- A real Uvicorn server returned ready health and a prediction matching the supplied reference; the temporary test server was then stopped.
- The local Docker command is installed, but its Linux engine was not running. The image build/container run therefore remains **unverified**.
- macOS setup and execution have not been run here.

Do not claim a platform or container was tested based solely on instructions.